Fundamentals of Data Structures

Prof. Pranjal Pandit

Unit 1 Introduction to Algorithms and Data Structures

- From Problem to Program (Problem, Solution, Algorithm, Data Structure and Program).
- Data, Information, Knowledge and Data Structure;
- Abstract Data Types (ADT);
- Classification (Linear/Non-linear, Static/Dynamic, Persistent/Ephemeral).
- Algorithms: problem solving, characteristics, design tools (pseudo-code, flowchart).
- Complexity: space and time complexity, asymptotic notation (Big-O, Theta, Omega),
- Step-count method, analysis of Linear, Quadratic, Cubic and Logarithmic constructs.
- Algorithmic strategies: Divide and Conquer, Greedy.

Abstract Data Type

An **Abstract Data Type (ADT)** is a logical description of a data type that specifies:

- What data is stored
- What operations can be performed on the data
- Not how the data is stored or implemented

In simple words:

An ADT tells us *what* we can do with the data, but not *how* those operations are performed.

Example: List ADT

Suppose you have a list of student names.

Student List = [Amit, Neha, Rahul]

The **List ADT** allows the following operations:

- **Insert** a new student.
- **Delete** a student.
- **Search** for a student.
- **Update** a student's name.
- **Display** all students.

The ADT only tells us these operations are possible. It **does not tell us** whether the list is stored using:

- an array,
- a linked list,
- or any other implementation.

Real-Life Analogy

Think of an **ATM**.

You know you can:

Check your balance

Withdraw money

Deposit money

You **do not need to know** how the bank's software or database works internally.

Similarly, an **ADT** defines **what operations are available**, while hiding **how they are implemented**.

Definition for Exams

An Abstract Data Type (ADT) is a logical model of a data structure that defines the data and the operations that can be performed on it without specifying how those operations are implemented.

ADT vs Data Structure

Abstract Data Type (ADT)	Data Structure
Defines what operations can be performed	Defines how the data is organized and stored
Logical view	Physical implementation
Hides implementation details	Shows implementation details
Example: List ADT	Example: Array, Linked List

Classification of Data Structures

Data structures can be classified in three common ways:

- **Linear vs Non-linear** (based on arrangement of data)
- **Static vs Dynamic** (based on memory allocation)
- **Persistent vs Ephemeral** (based on how updates affect old data)

Linear Data Structures

Definition

A **linear data structure** stores data elements **in a sequential order**, where each element is connected to the previous and next element (except the first and last).

10 → 20 → 30 → 40 → 50

Each element has only one predecessor and one successor.

Characteristics

- Data is arranged one after another.
- Easy to traverse from beginning to end.
- Suitable for simple applications.

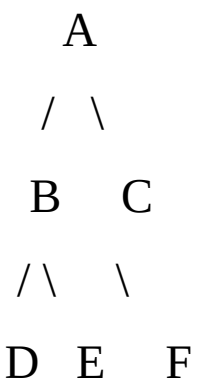
Examples

Data Structure	Example
Array	Student marks
Linked List	Music playlist
Stack	Stack of plates
Queue	People standing in a line

Non-linear Data Structures

Definition

A **non-linear data structure** stores data in a hierarchical or interconnected manner instead of a straight line.



One element can be connected to multiple elements.

Characteristics

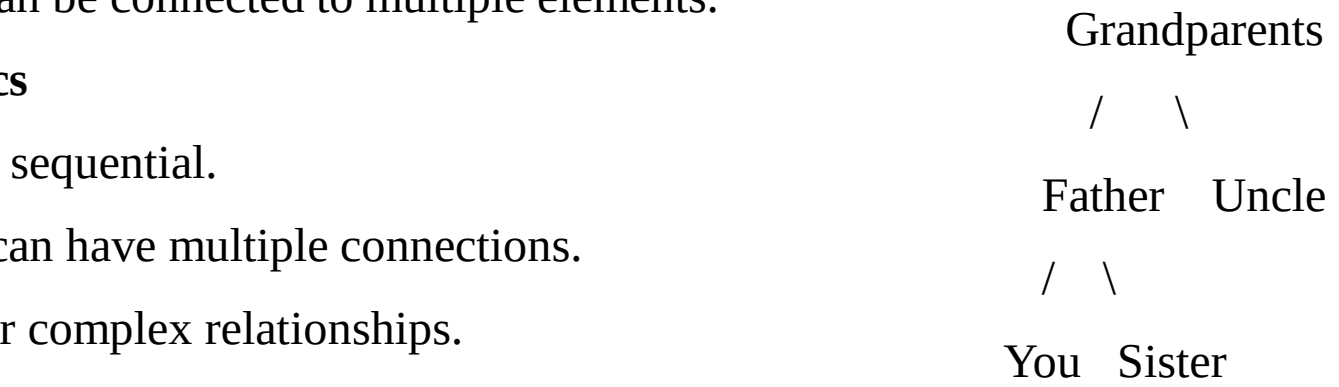
- Data is not sequential.
- One node can have multiple connections.
- Suitable for complex relationships.

Examples

Data Structure	Example
Tree	Organization hierarchy
Graph	Social media friendships

Real-life Example

Family Tree



Here, one person can have multiple children.

Difference: Linear vs Non-linear

Linear	Non-linear
Data arranged in a sequence	Data arranged hierarchically or as a network
One-to-one relationship	One-to-many or many-to-many relationship
Easier to traverse	More complex traversal
Example: Array, Linked List	Example: Tree, Graph

Static Data Structures

Definition

A **static data structure** has a **fixed size** decided before the program runs. Its size cannot be changed during execution.

Characteristics

- Memory allocated once.
- Size cannot increase or decrease.
- Faster access.

Example

- Array of size 5
- Index: 0 1 2 3 4
- Data : 10 20 30 40 50
- You cannot add a 6th element unless you create a new array.

Real-life Example

- A classroom with exactly **40 seats**.
- If a 41st student comes, there is no space.

Dynamic Data Structures

Definition

A **dynamic data structure** can **grow or shrink during program execution** as needed.

Characteristics

- Memory allocated at runtime.
- Flexible size.
- Better memory utilization.

Examples

- Linked List
- Tree
- Graph

Example

Initially

10 → 20 → 30

Insert 40

10 → 20 → 30 → 40

The structure grows without creating a new one.

Real-life Example

A shopping cart on an online shopping website. You can add or remove items anytime.

Difference: Static vs Dynamic

Static	Dynamic
Fixed size	Variable size
Memory allocated before execution	Memory allocated during execution
Less flexible	More flexible
Example: Array	Example: Linked List

5. Ephemeral Data Structures

Definition

An **ephemeral data structure** is one where **updates modify the original data directly**. After a change, the old version is lost.

Example

Initial list

[10, 20, 30]

Insert 40

[10, 20, 30, 40]

The previous version ([10, 20, 30]) no longer exists.

Real-life Example

- A notebook.
- If you erase a sentence and write a new one, the old sentence is gone.
- Most common data structures (arrays, linked lists, stacks, queues) are **ephemeral**.

6. Persistent Data Structures

Definition

A **persistent data structure** preserves the **old version** whenever it is modified. Every update creates a **new version**, while previous versions remain available.

Example

Version 1

[10, 20, 30]

Insert 40

Version 2

[10, 20, 30, 40]

Both versions exist.

Real-life Example

- **Google Docs Version History** or **Git (Version Control)**.
- Every time you save changes, older versions are still available.

Difference: Persistent vs Ephemeral

Persistent	Ephemeral
Old versions are preserved	Old versions are lost after updates
Every update creates a new version	Original data is modified
Used in version control systems, backups	Used in most normal programs
Example: Git, version history	Example: Array, Linked List

Summary Table

Classification	Type	Meaning	Example
Arrangement of Data	Linear	Sequential organization	Array, Linked List
	Non-linear	Hierarchical or network organization	Tree, Graph
Memory Allocation	Static	Fixed size	Array
	Dynamic	Variable size	Linked List
Update Behavior	Ephemeral	Original data is modified	Array, Linked List
	Persistent	Old versions are preserved	Git, Version History

Easy way to remember

- **Linear vs Non-linear → How is the data organized?**
- **Static vs Dynamic → Can the size change?**
- **Persistent vs Ephemeral → What happens to the old data after an update?**